

Actividad 4. Buenas Prácticas del EDA con Meta AI

Prompt 1. ¿Por qué la reproducibilidad es considerada uno de los pilares fundamentales de un proyecto de Ciencia de Datos profesional? ¿Cuáles son las principales prácticas para garantizarla en un notebook de Google Colab o Jupyter, y qué consecuencias tiene para un equipo de trabajo cuando un análisis no puede ser reproducido?

La reproducibilidad es el "control de calidad" de la Ciencia de Datos. Si otro científico de datos —o tú mismo en 6 meses— no puede llegar al mismo resultado con los mismos datos y código, entonces tu análisis vale tanto como una anécdota. En un proyecto profesional, eso es inaceptable.

1. ¿Por qué es un pilar fundamental?

Razón	Qué pasa si NO la tienes
Confianza	Stakeholders, auditores y reguladores no pueden confiar en decisiones basadas en números que "nadie sabe cómo salieron"
Debugging y mantenimiento	Un bug encontrado 3 meses después es imposible de rastrear si el notebook original ya no corre igual
Colaboración	El traspaso entre data scientists se rompe. "Funciona en mi máquina" se vuelve el estándar
Cumplimiento legal	Sectores como finanzas, salud y pharma exigen auditorías. GDPR, HIPAA, SOX te piden demostrar cómo llegaste a una conclusión
Ciencia	Sin reproducibilidad no es ciencia, es un PowerPoint caro. Los resultados deben verificarse por terceros

Razón	Qué pasa si NO la tienes
Iteración	No puedes mejorar un modelo si no puedes recrear su línea base. Te quedas estancado

2. Prácticas clave para garantizarla en Jupyter/Colab

Colab y Jupyter son geniales para explorar, pero son "fábricas de caos" si no pones reglas. Estas 7 prácticas te salvan:

A nivel de código y ejecución

1. Fija las semillas aleatorias: Todo lo que

use `random`, `numpy`, `torch`, `sklearn` necesita `random_state`.

Python

```
import numpy as np, random, os
SEED = 42
np.random.seed(SEED); random.seed(SEED);
os.environ['PYTHONHASHSEED'] = str(SEED)
```

1. Corre el notebook de arriba a abajo: Antes de entregar, `Kernel -> Restart & Run All`. Si explota, tu notebook no es reproducible. Elimina ejecuciones fuera de orden.

2. No uses variables globales mutables: Evita celdas que sobrescriben `df` 5 veces. Mejor `df_raw -> df_clean -> df_features`. Cada paso debe ser idempotente.

A nivel de entorno

4. Versiona dependencias: En Colab/Jupyter, el mayor enemigo son las versiones.

Python

```
# Al inicio del notebook !pip freeze > requirements.txt # o usa %pip install
pandas==2.1.4
```

Mejor aún: usa `conda env export` o `poetry` si corres local, y en Colab documenta las versiones clave con `pandas.**version**`.

5. Fija versiones de datos: Nunca leas `df = pd.read_csv('datos.csv')` de tu Drive sin versión. Usa DVC, una fecha en el nombre `ventas_2026-06-19.parquet`, o un commit hash. Los datos también cambian.

A nivel de proyecto

6. **Modulariza fuera del notebook:** Los notebooks son para narrar, no para definir funciones core. Mueve funciones a `src/utils.py` e importa. Así controlas versiones con git.

7. **Documenta supuestos y fuentes:** ¿Filtraste outliers? ¿Cómo imputaste nulos? ¿De dónde vino la tabla? Si no está escrito en markdown, no existe.

Usa `assert` para validar supuestos: `assert df['edad'].min() >= 0`

Extra para Colab

- **Fija la versión del runtime:** Colab actualiza Python y librerías sin avisar. Anota la versión de Python en el README.
- **No dependas del estado del filesystem:** `/content` se borra. Monta Drive o baja de GCS/S3 con credenciales versionadas.
- **Guarda artefactos:** Modelos, gráficos y tablas importantes → `joblib.dump(model, 'modelo_v1.pkl')` + súbelo a un bucket.

3. Consecuencias cuando un análisis NO es reproducible

Para un equipo, es como construir sobre arena movediza:

1. **Pérdida de tiempo brutal:** "¿Por qué el AUC bajó de 0.91 a 0.84?" El equipo gasta 2 semanas intentando recrear el 0.91 que salió por una semilla random + un filtro que alguien borró.
2. **Desconfianza entre áreas:** Producto/Finanzas deja de creer en Data. Si no puedes explicar por qué el forecast cambió, te sacan del roadmap.
3. **Riesgo de negocio y legal:** Tomaste una decisión de \$2M basada en un análisis que el auditor pide recrear. Si no puedes, hay multas o demandas.
4. **Onboarding imposible:** Un nuevo integrante abre el notebook, corre todo, y le dan resultados distintos. Su primer PR se retrasa 1 mes.
5. **Deuda técnica exponencial:** Empiezas a duplicar notebooks "por si acaso": `analisis_final.ipynb`, `analisis_final_v2.ipynb`, `analisis_final_AHORA_SI.ipynb`. Nadie sabe cuál es la verdad.

Regla de oro: Si no puedes borrar tu notebook, clonar el repo en una máquina limpia y obtener el mismo output con 1 comando, no es profesional todavía.

Conclusión: Al igual que en los experimentos científicos, los resultados deben de ser reproducibles para su validación y reconocimiento a nivel científico. La ausencia de reproducibilidad provoca diversos problemas a nivel de equipo y reputación del equipo o persona que lo realiza. Esto puede provocar diversos problemas pero el que considero mas importante a nivel general es la pérdida de tiempo pues al tomarse decisiones con un análisis no reproducible se está arriesgando el progreso, inversión, etc. de los recursos de la organización que lo realiza.

Prompt 2. ¿Cuál es la diferencia entre documentar el qué y documentar el por qué en un notebook de análisis de datos? Proporciona un ejemplo concreto de una decisión analítica (como la elección entre media o mediana para imputar) y muestra cómo debería documentarse correctamente tanto en una celda Markdown como en un comentario de código.

En un notebook de análisis de datos, documentar el *qué* vs documentar el *por qué* es la diferencia entre que alguien entienda tu código en 6 meses... o que te quiera quemar vivo.

La diferencia clave

	Documentar el QUÉ	Documentar el POR QUÉ
Define	Lo que hace el código	La razón detrás de la decisión
Responde	¿Qué pasa aquí?	¿Por qué lo hiciste así?
Nivel	Mecánico, obvio leyendo el código	Contexto, trade-offs, intención
Audiencia	Cualquiera que sepa Python/pandas	Tu yo del futuro y tus colegas
Regla	Si el código ya lo dice, no lo repitas	Si la razón no es obvia, escríbela

Regla de oro: El código muestra el *qué*. Los comentarios y Markdown explican el *por qué*. Si tu comentario solo traduce el código a español, bórralo.

Ejemplo concreto: Imputar con media vs mediana

Contexto: Tienes una columna `edad_cliente` con 8% de valores nulos. Decides imputar.

✗ Mal documentado - Solo el QUÉ

Celda de código:

Python

```
# Llenar nulos con la mediana
df['edad_cliente'] = df['edad_cliente'].fillna(df['edad_cliente'].median())
```

Celda Markdown:

Code

```
### Imputación de edad_cliente
Se rellenan los valores nulos usando la mediana.
```

Problema: Cualquiera ve que usaste la mediana. Nadie sabe por qué no usaste la media, KNN, o por qué no tiraste las filas.

✓ Bien documentado - QUÉ + POR QUÉ

Celda Markdown - Contexto y decisión:

3.2 Imputación de `edad_cliente`

Problema: 8% de los registros tienen edad nula. Necesitamos imputar porque `edad_cliente` es feature clave para el modelo de churn.

Decisión: Imputamos con la mediana en lugar de la media.

Justificación:

1. **Distribución sesgada:** La edad tiene cola a la derecha por clientes 65+. La media = 42.3 años, mediana = 38 años. Usar la media inflaría artificialmente la edad de los nulos.
2. **Outliers:** Tenemos 12 clientes >90 años que sesgan la media. La mediana es robusta a estos casos.
3. **Impacto en modelo:** Árboles de decisión son poco sensibles a imputación, pero regresión logística sí. La mediana reduce el riesgo de crear un sesgo artificial hacia clientes mayores.

Alternativas descartadas:

- Eliminar filas: perderíamos 8% de datos = 12k clientes
- KNN Imputer: costoso computacionalmente y `edad` no correlaciona fuerte con otras variables

Celda de código - Comentario con el POR QUÉ específico:

Python

```
# POR QUÉ mediana: edad tiene distribución sesgada a la derecha + outliers >90 años# La media de 42.3 sobrestimaría vs mediana de 38. Ver histograma en celda 3.1# Trade-off: perdemos varianza, pero evitamos sesgo hacia clientes mayoresdf['edad_cliente'] = df['edad_cliente'].fillna(df['edad_cliente'].median())
# Validación: verificamos que la distribución post-imputación mantiene forma similarassert abs(df['edad_cliente'].skew() - skew_antes) < 0.1, "Imputación distorsionó el skew"
```

2 líneas ocultas

Plantilla que puedes usar siempre

Para Markdown antes del código:

[Nombre del paso]

Objetivo: Qué intentas lograr con este bloque

Decisión: Qué enfoque elegiste

Por qué:

1. Razón técnica 1: datos, distribución, performance
2. Razón de negocio 2: impacto, supuestos del dominio
3. Razón 3: qué probaste y descartaste

Riesgos/Trade-offs: Qué problemas podría causar esta decisión

Para comentarios en código:

Python

```
# POR QUÉ [decisión]: [razón técnica en 1 línea]# Trade-off: [qué sacrificas] para ganar [qué obtienes]# Ref: [link a ticket, paper, o celda con análisis exploratorio]codigo_que_hace_el_que()
```

Otros ejemplos rápidos de POR QUÉ

Decisión	Mal comentario - solo QUÉ	Buen comentario - POR QUÉ
<code>df.drop_duplicates()</code>	Eliminar duplicados	POR QUÉ drop: API devuelve duplicados si usuario recarga página. Mantenemos primera ocurrencia por <code>timestamp</code> porque es el registro original
<code>pd.to_datetime(errors='coerce')</code>	Convertir a fecha	POR QUÉ coerce: 0.3% de fechas vienen como 'N/A' de la fuente. Coerce → NaT evita que crashee el pipeline. Se imputan después en 4.2
<code>df = df[df['ventas'] > 0]</code>	Filtrar ventas > 0	POR QUÉ > 0: ventas=0 son cancelaciones y ventas<0 son devoluciones. Modelo predice solo ventas efectivas según definición de Negocio

El test definitivo: Si borras todos los comentarios y celdas Markdown, ¿alguien podría reconstruir tu razonamiento viendo solo el código? Si la respuesta es no, te falta documentar el *por qué*.

Conclusión: La documentación en cualquier proceso computacional es vital para la claridad del código, tanto para el autor como para terceros. Expandiendo la documentación a procesos mas generales dentro de la investigación su importancia radica en la posible continuidad o consulta de los resultados o procedimientos de la investigación concretada.

Prompt 3. ¿Cómo se estructura un Análisis Exploratorio de Datos (EDA) en un equipo profesional de Data Science? ¿Qué entregables se esperan al finalizar el EDA antes de pasar a la fase de modelado, y qué herramientas o formatos (como reportes, dashboards o notebooks) se utilizan comúnmente para comunicar los hallazgos a stakeholders no técnicos?

Un **Análisis Exploratorio de Datos (EDA)** en un equipo profesional de Data Science no es solo "hacer gráficos". Es un proceso estructurado para entender los datos, detectar riesgos y definir la estrategia de modelado. Se hace para responder: *¿Podemos confiar en estos datos para resolver el problema de negocio?*

Aquí va el flujo completo que se usa en equipos de nivel senior:

1. Estructura estándar de un EDA profesional

Se divide en 6 fases. Cada una responde una pregunta clave:

Fase	Objetivo	Preguntas que responde	Output técnico
1. Contexto y Definición	Alinear con el problema de negocio	¿Cuál es la variable target? ¿Qué métrica de negocio importa? ¿Qué hipótesis tenemos?	Documento de 1 página: "EDA Charter"
2. Auditoría de Calidad	Medir qué tan sucios están los datos	% missing, duplicados, outliers, cardinalidad, data drift vs producción	Tabla de calidad + Data Profile
3. Análisis Univariado	Entender cada variable por separado	Distribución, sesgo, rangos, categorías dominantes, fugas de información	Histogramas, bar plots, summary stats
4. Análisis Bivariado/Multivariado	Encontrar relaciones con el target	¿Qué features predicen mejor? ¿Hay colinealidad? ¿Interacciones?	Matriz de correlación, feature importance inicial, scatter plots
5. Validación de Hipótesis	Testear supuestos del negocio	"Los usuarios premium gastan más los viernes" → ¿Es cierto?	Pruebas estadísticas: t-test, chi2, ANOVA

Fase	Objetivo	Preguntas que responde	Output técnico
6. Conclusiones y Riesgos	Traducir a decisiones	¿Qué datos faltan? ¿Hay data leakage? ¿Qué features crear? ¿Podemos modelar?	Go/No-Go para modelado + backlog de features

Regla de oro: 70% del tiempo de un EDA serio se va en fase 1 y 2. Si te saltas la calidad, el modelo falla en producción.

2. Entregables esperados antes de modelar

Un PM/Lead de DS no te dejará tocar un **XGBoost** sin estos 4 entregables firmados:

1. Data Quality Report:

- **Qué contiene:** % nulos por columna, % outliers, distribución vs datos históricos, esquema de datos.
- **Formato:** Tabla con semáforo verde/amarillo/rojo por variable.
- **Decisión:** "Columna **income** tiene 43% nulos → la descartamos o imputamos".

2. Feature Insights Document:

- **Qué contiene:** Top 10 variables con mayor correlación al target, hallazgos contraintuitivos, riesgos de leakage.
- **Formato:** 1 slide por insight clave. Ej: "Clientes con 0 logins en 90 días tienen 8x más churn".
- **Decisión:** Define el primer set de features para el baseline.

3. EDA Summary Deck:

- **Qué contiene:** 5-8 slides para stakeholders no técnicos. 0 código.
- **Estructura:** Problema → Datos usados → 3 hallazgos clave → Riesgos → Recomendación: Go/No-Go.
- **Decisión:** El negocio aprueba seguir invirtiendo tiempo de ML.

4. Contrato de Datos para Modelado:

- **Qué contiene:** Definición final de train/test split, tratamiento de nulos, cómo manejar outliers, lista negra de columnas con leakage.
- **Formato:** README.md en el repo + ticket de Jira/Asana.
- **Decisión:** Evita que cada DS modele con supuestos distintos.

Si no hay Go/No-Go documentado, el EDA no terminó.

3. Herramientas y formatos para comunicar a stakeholders no técnicos

El error #1 es mandar un notebook de 200 celdas al director de Marketing. Adapta el formato a la audiencia:

Audiencia	Formato preferido	Herramienta	Qué enfatizar
Ejecutivos / PMs	Deck de 5 slides	Google Slides, PowerPoint, Canva	Impacto en \$/%, riesgos, decisión. 1 gráfico gigante por slide.
Product / Negocio	Dashboard interactivo	Streamlit, Dash, Tableau, Looker	Que puedan filtrar ellos mismos. "Muéstrame el churn por ciudad"
Equipo Técnico DS	Notebook documentado	Jupyter, Quarto, Databricks	Código reproducible, tests, profiling automático
Toda la org	Reporte auto-generado	ydata-profiling, sweetviz, DataDoc	One-pager de salud de datos que se corre cada semana

Stack más común en 2026:

1. **Exploración:** pandas, polars, duckdb para velocidad + ydata-profiling para auto-EDA inicial.
2. **Visualización:** plotly y altair porque son interactivos → se embeben en slides/dashboards.
3. **Comunicación:**
 - Quarto o JupyterBook: Convierte tu notebook en web/reporte con 1 comando. Ideal para técnicos.

- **Streamlit**: Haces una app en 50 líneas de Python. El estándar para demos a negocio.
- **Hex** o **Deepnote**: Notebooks colaborativos con modo "app" para stakeholders.

4. Errores que delatan un EDA junior

1. **"Encontré que X se correlaciona con Y"** sin checar si X está disponible en inferencia. Eso es leakage.
2. **Mostrar p-values** a un director. Tradúcelo: "Con 95% de confianza, los usuarios de iOS gastan \$12 más".
3. **No cuantificar el impacto**: "Hay nulos" → junior. "El 30% de nulos nos hace perder \$200k de revenue potencial" → senior.
4. **EDA sin siguiente paso**: Todo hallazgo debe terminar en "Por eso vamos a hacer X en el modelo".

Pro tip: Antes de empezar el EDA, pregunta: *¿Qué decisión tomaremos si encontramos A vs B?* Si la respuesta es "ninguna", ese análisis no se hace.

Conclusiones: La estandarización de este tipo de procesos es de vital importancia pues permite obtener un mismo flujo de trabajo en diferentes centros de investigación, empresas, etc.